

Chapter 16. 품질 기준을 프로젝트마다 다르게 — 설정 가능한 Harness

이 챕터에서 배우는 것

- Gate 가중치가 왜 CLI 플래그가 아니라 `.env` 로 노출됐는지
- "죽은 파라미터"가 무엇이었고 어떻게 실제로 배선됐는지
- SDK가 이미 제공하는 옵트인 파라미터를 "그냥 안 켜고 있었던" 사례가 실제로 있었다는 것
- 관대한 파싱(값이 잘못돼도 전체를 죽이지 않는) 원칙이 여기서 어떻게 적용되는지

이런 분이 먼저 읽으면 좋습니다: 9장에서 본 Gate A의 가중 평균 공식($0.4 \times \text{TCR} + 0.6 \times \text{나머지}$)이 고정값처럼 보였는데, 실제로 조정할 방법이 있는지 궁금한 분.

16.1 발견 — 이미 있던 파라미터가 아무도 안 쓰고 있었다

`eval/monitor.py` 의 `build_book_monitor()` 는 처음부터 `enable_llm_judge` / `judge_model` 파라미터를 갖고 있었다. 그런데 실제로 확인해 보니, `new` · `draft` · `chat` · `build` · `research` · `plan` · `review` 7개 CLI 명령 **어느 하나도 이 값을 실제로 넘기지 않았다**. 항상 기본값(꺼짐)만 쓰였다. 함수 시그니처에는 있지만 실질적으로 절대 켜지지 않는 **죽은 파라미터**였다. Gate 가중치 (`gate_a_tcr_weight` 등, SDK가 이미 지원)도 마찬가지로 어디에도 노출돼 있지 않았다.

이 발견은 이 챕터 전체의 출발점이다 — "기능이 코드에 존재한다"는 것과 "그 기능이 실제로 도달 가능하다"는 것은 다른 문제다.

16.2 배선 — CLI 플래그로 LLM Judge를 옵트인시킨다

파일: `src/book_forge/cli/commands/new_cmd.py`

```
@click.option(
    "--enable-llm-judge", is_flag=True,
    help="일부 샘플에 LLM 채점(faithfulness 등)을 추가 적용 (기본 off - 비용/지연 증가)",
)
@click.option(
    "--judge-model", default=None,
    help="[--enable-llm-judge] 채점에 쓸 모델 (미지정 시 API 키 기반 자동 결정)",
)
def new(..., enable_llm_judge: bool, judge_model: Optional[str]) → None:
    ...
    monitor = build_book_monitor(
        output_dir=str(project_dir / "eval_results"),
        enable_llm_judge=enable_llm_judge,
        judge_model=judge_model,
    )
```

새 판정 로직은 전혀 없다. 이미 있던 파라미터를 CLI 플래그까지 실제로 연결했을 뿐이다. `draft_cmd.py` 도 같은 방식으로 배선됐다. `enable_llm_judge` 가 기본 off인 이유는 명확하다. Agent-Evaluator의 `LLMJudge` 는 OpenAI/Anthropic 모델만 지원한다(Ollama 연동이 없다, `grep` 으로 확인). 1장에서 다룬 "기본 provider는 Ollama, API 키 없이 동작"이라는 원칙과 이 옵션은 정면으로 부딪힌다. 그래서 명시적으로 켜야만 API 키를 요구하는 경로로 들어간다.

16.3 Gate 가중치 — 왜 CLI 플래그가 아니라 `.env` 인가

10장이 다룬 방법론(실패 모드에서 Gate를 도출하고, 회복 난이도·발생 빈도·사용자 가시성으로 가중치를 정하는 것)이 "무엇을, 왜"를 답했다면, 이 절은 그 결정을 실제 코드로 어떻게 조정 가능하게 만들었는지 — "어떻게"를 답한다.

파일: `src/book_forge/eval/monitor.py`

```
_GATE_WEIGHT_ENV_VARS = (  
    ("BOOK_FORGE_GATE_A_TCR_WEIGHT", "gate_a_tcr_weight"),  
    ("BOOK_FORGE_GATE_C_TCR_WEIGHT", "gate_c_tcr_weight"),  
    ("BOOK_FORGE_GATE_B_LOOP_WEIGHT", "gate_b_loop_weight"),  
)  
  
def _gate_weight_overrides() → dict[str, float]:  
    overrides: dict[str, float] = {}  
    for env_name, kwarg_name in _GATE_WEIGHT_ENV_VARS:  
        raw = os.environ.get(env_name)  
        if raw is None:  
            continue  
        try:  
            overrides[kwarg_name] = float(raw)  
        except ValueError:  
            continue  
    return overrides
```

Gate 가중치는 매번 명령을 입력할 때마다 타이핑하는 값이 아니라 "이 환경에서는 항상 이 기준으로 판정한다"는 **저자별 고정 설정**에 가깝다. 그렇게 판단해 `.env` 로 노출했다 — 1장에서 다룬 `LLM_PROVIDER` 선택과 정확히 같은 성격이다.

`build_book_monitor()` 가 이 값을 `**_gate_weight_overrides()` 로 `PerformanceMonitor` 에 그대로 펼쳐 넘긴다.

파일: `src/book_forge/eval/monitor.py` (`build_book_monitor()`)

```
return PerformanceMonitor(  
    output_dir=output_dir,  
    enable_security_metrics=True,  
    enable_hallucination_detection=False,  
    enable_llm_judge=enable_llm_judge,  
    judge_model=judge_model,  
    judge_sample_rate=0.2,  
    use_korean_tokenizer=True,  
    auto_save=True,  
    auto_save_interval=5,  
    **_gate_weight_overrides(),  
)
```

16.4 use_korean_tokenizer — Book-forge 산출물의 언어에 맞춰 SDK를 조정한다

`use_korean_tokenizer=True` 는 이 책을 준비하며 실측 감사 과정에서 새로 추가된 플래그다. 그 전까지 Book-forge는 이 값을 지정하지 않아 SDK 기본값(`False`)을 그대로 썼다. Agent-Evaluator SDK의 `PerformanceMonitor` docstring이 이 값의 효과를 정확히 밝힌다.

```
" use_korean_tokenizer : True 이면 AccuracyEvaluator /
HallucinationDetector 에서 kiwipiepy 형태소 분석기를 사용한다( pip install
"agent-evaluator[korean]" 필요). 미설치 시 공백 분리 풀백으로 동작하며 경고 로
그가 출력된다."
```

Book-forge의 산출물은 예외 없이 한국어다. 그런데 Gate A(`AccuracyEvaluator`)와 Gate C(`HallucinationDetector`)의 기본 단어 비교 방식은 공백 분리(whitespace split) 토큰화다. 한국어는 조사·어미가 단어에 붙는 교착어라 공백 분리만으로는 "코드베이스를" 과 "코드베이스가"를 다른 단어로 취급하는 등 실제 의미 일치를 놓치기 쉽다. `kiwipiepy` 형태소 분석기를 쓰면 이런 경우도 같은 명사(어간)로 인식해 정확도 채점이 더 정밀해진다.

이 플래그를 켜는 것이 안전한 이유는 SDK 자신의 풀백 설계에 있다. `kiwipiepy`가 설치돼 있지 않으면 예외를 던지는 대신 **공백 분리로 자동 풀백**하고 경고 로그만 남긴다(이 챕터가 반복해온 "관대한 파싱" 원칙과 정확히 같은 자리에 있다). 그래서 이 값은 무조건 `True`로 요청해도 안전하다. `kiwipiepy`가 있으면 더 정확해지고, 없으면 기존과 동일하게 동작할 뿐이다. Book-forge는 `pyproject.toml`에 `korean = ["agent-evaluator[korean]"]`라는 옵트인 extra를 추가해, 이 정확도 향상을 원하는 사용자가 `pip install -e "[korean]"`으로 선택할 수 있게 해줬다.

 **개발자 TIP:** 이 발견은 "Agent-Evaluator SDK가 이미 제공하는 기능인데 Book-forge가 그동안 켜지 않고 있었다"는 점에서 12장 (§ 12.2)의 `PerformanceMonitor.merge()` / `load_from_file()` 사례, 15장 (§ 15.4)의 `.aoo/claims.jsonl` 사례와 같은 계보다. Book-forge와 Agent-Evaluator를 함께

쓸 때는 "새 기능을 요청하기 전에, 이미 있는 옵션인 파라미터부터 켜봤는가"를 먼저 점검할 가치가 있다.

16.5 관대한 파싱 — 오타 하나가 파이프라인을 죽이면 안 된다

`_gate_weight_overrides()`의 `try/except ValueError: continue`는 이 코드베이스에서 반복되는 원칙을 그대로 따른다. 함수 docstring이 명시한다.

"값이 float으로 파싱 안 되면(오타 등) 그 항목만 조용히 무시한다 — 다른 `alternative_suggester.py`의 '관대한 파싱, 실패해도 결과는 낸다' 원칙과 동일(잘못된 설정 하나 때문에 전체 파이프라인이 죽으면 안 됨)."

`.env`에 `BOOK_FORGE_GATE_A_TCR_WEIGHT=zero point four`처럼 잘못 쓰더라도, 그 한 줄 때문에 `book-forge new` 전체가 예외로 죽지 않는다. 그 항목만 무시하고 `PerformanceMonitor`의 원래 기본값(`gate_a_tcr_weight=0.4`)으로 조용히 되돌아간다. 이는 5장 (§ 5.4)에서 다룬 "관대한 파싱"과 "안전한 폴백"(형식을 어겨도 예외 대신 미리 정한 안전한 값으로 되돌아가는) 원칙이 설정 값 파싱에도 일관되게 적용된 사례다.

 **QA 관리자 TIP:** 이 설계는 "설정이 잘못돼도 조용히 기본값으로 넘어간다"는 뜻이기도 하다. 오타를 낸 걸 사용자가 알아채지 못할 수 있다는 트레이드오프가 있다. Book-forge 전체가 "파이프라인을 절대 죽이지 않는다"는 방향을 일관되게 택하고 있다는 것을 이해하고 나면, 설정이 의도대로 반영됐는지 별도로 확인하는 습관 (`PerformanceMonitor` 인스턴스의 실제 속성값을 로그로 남기는 등)이 왜 필요한지 알 수 있다.

16.6 실측 — `.env` 값이 실제로 반영되는지 API 키 없이 확인 가능하다

Gate 가중치 조정은 API 키가 필요 없는 경로다. `.env` 파일에

`BOOK_FORGE_GATE_A_TCR_WEIGHT=0.9` 를 쓰면, 실제 `load_config()` (python-dotenv) → `os.environ` → `PerformanceMonitor(gate_a_tcr_weight=0.9)`

전체 경로가 몽키패치 없이 실측으로 확인됐다. `.env` 에 쓴 값이 실제로

`PerformanceMonitor` 인스턴스의 내부 속성에 그대로 반영됨을 직접 실행으로 검증할 수 있었다.

직접 해보기

프로젝트 디렉토리(또는 저장소 루트)의 `.env` 에

`BOOK_FORGE_GATE_A_TCR_WEIGHT=0.9` 를 써넣고 `book-forge gate` 를 다시 실행해보라. 같은 태스크 데이터인데 Gate A 점수가 달라지는 것을 직접 확인할 수 있다 (§ 16.5). 값을 `zero point four` 처럼 일부러 잘못 써넣어도 전체 파이프라인이 죽지 않고 그 항목만 조용히 기본값으로 돌아간다는 것(§ 16.4 관대한 파싱)도 함께 확인해보라.

여러 환경(로컬/CI/프로덕션)에서 다른 품질 기준이 필요한 프로젝트라면 기억해둘 규칙 하나 — 매번 CLI 인자로 넘기는 대신 환경변수 방식이 "이 환경에서는 항상 이 기준"이라는 의도를 더 정확히 코드로 표현한다.

이 챕터의 핵심

- **파라미터가 시그니처에 있다고 실제로 도달 가능한 것은 아니다.**

`enable_llm_judge` 가 7개 CLI 명령 어디서도 실제로 안 쓰이던 "죽은 파라미터"였던 사례가 이를 보여준다.

- **설정을 CLI 플래그로 둘지 `.env` 로 둘지는 "매번 바뀌는가, 환경마다 고정인가"로 결정한다.** LLM Judge는 옵트인 플래그, Gate 가중치는 `.env`.
- **SDK의 옵트인 파라미터를 안 켜고 있는 것도 "죽은 파라미터"의 또 다른 형태다.** `use_korean_tokenizer` 가 그 실제 사례 — 미설치 시 안전하게 폴백하는 파라미터는 기본적으로 켜두는 편이 낫다.

- **관대한 파싱은 설정 값에도 일관되게 적용된다.** 오타 하나가 전체 파이프라인을 죽이지 않는다.

참고 자료

- 부록 B.5(업계 동향) — LLM Judge의 신뢰성·편향 논쟁, `--enable-llm-judge`가 기본 off인 선택이 왜 최근 연구와도 맞물리는지
 - `src/book_forge/eval/monitor.py` — 전체
 - `src/book_forge/cli/commands/new_cmd.py` · `draft_cmd.py` — 실제 플래그 배선
 - `Book-forge/pyproject.toml` — `korean` extra(`agent-evaluator[korean]`)
 - `Agent-Evaluator/agent_evaluator/core/trackers/monitor.py` — `PerformanceMonitor.__init__()`의 `use_korean_tokenizer` 파라미터 docstring
-

마지막 챕터는 지금까지 다룬 모든 메커니즘의 경계 밖 — Book-forge가 아직 하지 못하는 것을 정직하게 다룬다.